

ASSIGNMENT-11.4

NAME: G. MANI SHARAN REDDY

BATCH: 41

HALL.NO: 2303A52148

Task 1: Stack for Undo Operations (LIFO)

Prompt Used

Implement a Stack class in Python with push, pop, peek, and is_empty methods. Add proper docstrings, explain why Stack is suitable for undo functionality, and compare list-based and deque-based implementations.

Code

```
43.py > ...
1  class Stack:
2      """
3      Stack implementation using Python list.
4      Follows LIFO (Last-In-First-Out) principle.
5      Suitable for undo operations.
6      """
7
8      def __init__(self):
9          """Initialize an empty stack."""
10         self.items = []
11
12     def push(self, action):
13         """
14         Push an action onto the stack.
15         Time Complexity: O(1)
16         """
17         self.items.append(action)
18
19     def pop(self):
20         """
21         Remove and return the most recent action.
22         Time Complexity: O(1)
23         """
24         if not self.is_empty():
25             return self.items.pop()
```

```

1  class Stack:
19     def pop(self):
25         return self.items.pop()
26         return None
27
28     def peek(self):
29         """
30         Return the top element without removing it.
31         Time Complexity: O(1)
32         """
33         if not self.is_empty():
34             return self.items[-1]
35         return None
36
37     def is_empty(self):
38         """Return True if stack is empty."""
39         return len(self.items) == 0
40
41
42     # Test Case: Simulating Undo Operations
43
44     stack = Stack()
45
46     stack.push("Type A")
47     stack.push("Type B")
48     stack.push("Delete B")
49
50     print("Undo:", stack.pop())
51     print("Current Top:", stack.peek())
52     print("Undo:", stack.pop())

```

Optimized

```

stack.push("Delete B")

print("Undo:", stack.pop())
print("Current Top:", stack.peek())
print("Undo:", stack.pop())
from collections import deque

✓ class StackDeque:
✓     def __init__(self):
✓         self.items = deque()

✓     def push(self, action):
✓         self.items.append(action)

✓     def pop(self):
✓         if self.items:
            return self.items.pop()
        return None
print("Stack using deque:")
stack_deque = StackDeque()
stack_deque.push("Type A")
stack_deque.push("Type B")
stack_deque.push("Delete B")

```

Output:

```
Undo: Delete B
Current Top: Type B
Undo: Type B

313/python.exe C:/Users/S9409/Downloads/a1ass1cantcoding/43.py
Undo: Delete B
Current Top: Type B
Undo: Type B
Stack using deque:
Undo: Delete B
Undo: Type B
Undo: Type A
```

Justification

Why Stack is Suitable for Undo

Undo operations must reverse the most recent action first. This matches LIFO behavior:

- Latest action → undone first
- Earlier actions → undone later

Example:

1. Type A
2. Type B
3. Delete B

Undo → Delete B undone first

Stack provides $O(1)$ push and pop, making it ideal for frequent undo operations.

Task 2: Queue for Customer Service (FIFO)

Prompt Used

Implement a Queue using list, analyze performance, and optimize using deque.

Code

```

.py / ...
class QueueList:
    """
    Queue implementation using Python list.
    Follows FIFO (First-In-First-Out).
    """

    def __init__(self):
        self.queue = []

    def enqueue(self, request):
        """
        Add request to end of queue.
        Time Complexity: O(1)
        """
        self.queue.append(request)

    def dequeue(self):
        """
        Remove first request.
        Time Complexity: O(n) due to shifting.
        """
        if not self.is_empty():
            return self.queue.pop(0)
        return None

    def is_empty(self):
        return len(self.queue) == 0

# Test Case
q1 = QueueList()
q1.enqueue("Customer1")
q1.enqueue("Customer2")
print("Serving:", q1.dequeue())  # Define a constant instead of duplicating this literal "Serving:" 4 times. [+3 locations]
print("Serving:", q1.dequeue())
print("Serving:", q1.dequeue())  # Should return None (empty queue)
print("Is queue empty?", q1.is_empty())
print("Enqueueing Customer3")
q1.enqueue("Customer3")
print("Serving:", q1.dequeue())
print("Is queue empty?", q1.is_empty())

```

Optimized

```

from collections import deque

class QueueDeque:
    """
    Optimized queue implementation using collections.deque.
    """

    def __init__(self):
        self.queue = deque()

    def enqueue(self, request):
        self.queue.append(request)  # O(1)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.popleft()  # O(1)
        return None

    def is_empty(self):
        return len(self.queue) == 0

# Test Case
q2 = QueueDeque()
q2.enqueue("Customer1")
q2.enqueue("Customer2")
print("Serving:", q2.dequeue())

```

Output:

```
513/python.exe C:/Users/S/35409/Doc
Serving: Customer1
Serving: Customer2
Serving: None
Is queue empty? True
Enqueueing Customer3
Serving: Customer3
Is queue empty? True
```

Performance Analysis

List-Based Queue

- enqueue $\rightarrow O(1)$
- dequeue $\rightarrow O(n)$ (shifts elements)

For 10,000 requests, dequeue becomes expensive.

Deque-Based Queue

- enqueue $\rightarrow O(1)$
- dequeue $\rightarrow O(1)$

Deque uses a doubly-linked structure internally, avoiding element shifting.

Real-World Impact

In high-volume systems (banking, server queues, chat systems), list-based queues degrade performance significantly. Deque ensures stable $O(1)$ operations.

Task 3: Singly Linked List for Dynamic Playlist Management

Scenario

You are designing a music playlist feature where songs can be added or removed dynamically while maintaining order.

Task Description

- Implement a Singly Linked List with:

- o insert_at_end(song)

- o delete_value(song)

- o traverse()

prompt used

Add inline comments explaining pointer manipulation, Highlight tricky parts of insertion and deletion, Suggest edge case test scenarios (empty list, single node, deletion at head)

Code

```
class Node:
    """
    Node class representing each song in playlist.
    """
    def __init__(self, song):
        self.song = song
        self.next = None # Pointer to next node

class SinglyLinkedList:
    """
    Singly Linked List for playlist management.
    """
    def __init__(self):
        self.head = None

    def insert_at_end(self, song):
        """
        Insert song at end of list.
        """
        new_node = Node(song)

        # Case 1: Empty list
        if self.head is None:
            self.head = new_node
            return

        # Traverse until last node
        current = self.head
        while current.next:
            current = current.next

        # Update last node pointer
        current.next = new_node

    def delete_value(self, song):
        """
        Delete first occurrence of song.
        """
        current = self.head
```

```

def delete_node(self, song):
    """Delete a node from the playlist"""
    if current and current.song == song:
        self.head = current.next
        return

    prev = None
    while current and current.song != song:
        prev = current
        current = current.next

    # Case 2: Song not found
    if current is None:
        return

    # Bypass node (pointer update)
    prev.next = current.next

def traverse(self):
    """Traverse and print playlist"""
    current = self.head
    while current:
        print(current.song, end=" -> ")
        current = current.next
    print("None")

print("Playlist using Singly Linked List:")
playlist = SinglyLinkedList()
playlist.insert_at_end("Song A")
playlist.insert_at_end("Song B") # Define a constant instead of duplicating this literal "Song B" 4 times. [+3 locations]
playlist.insert_at_end("Song C")
playlist.traverse()
playlist.delete_value("Song B")
playlist.traverse()
print("Playlist using built-in list:")
playlist_list = []
playlist_list.append("Song A")
playlist_list.append("Song B")
print("Playlist:", playlist_list)
playlist_list.remove("Song B")
print("Playlist after deletion:", playlist_list)

```

Output:

```

Playlist using Singly Linked List:
Song A -> Song B -> Song C -> None
Song A -> Song C -> None
Playlist using built-in list:
Playlist: ['Song A', 'Song B']
Playlist after deletion: ['Song A']

```

Edge Cases

- Empty list insertion
- Delete from empty list
- Delete head node
- Delete single node list
- Delete non-existent value

Task 4: Binary Search Tree for Fast Record Lookup

Scenario

You are building a student record system where quick searching by roll number is required.

Task Description

- Implement a Binary Search Tree (BST) with:
 - o insert(value)
 - o search(value)
 - o inorder_traversal()
- Provide AI with a partially written Node and BST class.

Prompt used :

Complete missing methods, Add meaningful docstrings, Explain how BST improves search efficiency over linear search

Code

```
1 class Node:
2     """
3     Node of Binary Search Tree.
4     """
5     def __init__(self, value):
6         self.value = value
7         self.left = None
8         self.right = None
9
10
11 class BST:
12     """
13     Binary Search Tree implementation.
14     """
15     def __init__(self):
16         self.root = None
17
18     def insert(self, value):
19         """
20         Insert value while maintaining BST property.
21         """
22
23     def _insert(node, value):
24         if node is None:
25             return Node(value)
26
27         if value < node.value:
28             node.left = _insert(node.left, value)
29         else:
30             node.right = _insert(node.right, value)
31         return node
32
33
```



```

        self.root = _insert(self.root, value)

    def search(self, value):
        """
        Search value in BST.
        """
        current = self.root

        while current:
            if value == current.value:
                return True
            elif value < current.value:
                current = current.left
            else:
                current = current.right

        return False

    def inorder_traversal(self):
        """
        Print values in sorted order.
        """
        def _inorder(node):
            if node:
                _inorder(node.left)
                print(node.value, end=" ")
                _inorder(node.right)

        _inorder(self.root)

print("BST Example:")
bst = BST()
bst.insert(5)
bst.insert(3)
bst.insert(7)
print("Inorder Traversal:", end=" ")
bst.inorder_traversal()
print("\nSearch for 3:", bst.search(3))
print("Search for 10:", bst.search(10))

```

Output:

```

BST Example:
Inorder Traversal: 3 5 7
Search for 3: True
Search for 10: False

```

Why BST Improves Search Efficiency

Approach	Time Complexity
Linear Search	$O(n)$
BST (Average)	$O(\log n)$
BST (Worst - Skewed)	$O(n)$

BST reduces search space by half at each step in balanced cases.

Task 5: Graph Traversal for Social Network Connections

Scenario

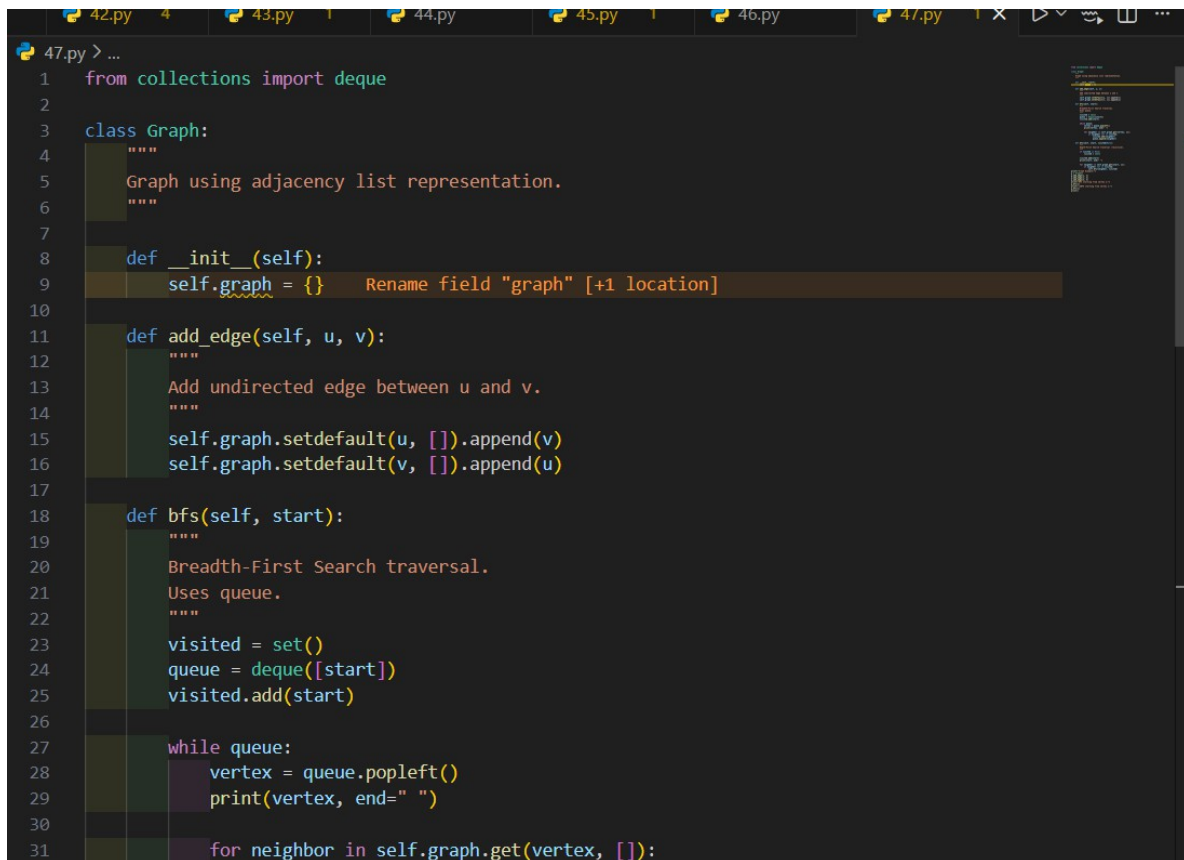
You are modeling a social network, where users are connected to friends, and you want to explore connections.

Task Description

- Represent the network using a graph (adjacency list).
- Use AI to implement:
 - o Breadth-First Search (BFS) to find nearby connections
 - o Depth-First Search (DFS) to explore deep connection paths

prompt used:

Add inline comments explaining traversal steps, Compare recursive and iterative DFS approaches, Suggest practical use cases for BFS vs DFS



```
47.py > ...
1  from collections import deque
2
3  class Graph:
4      """
5      Graph using adjacency list representation.
6      """
7
8      def __init__(self):
9          self.graph = {}  # Rename field "graph" [+1 location]
10
11     def add_edge(self, u, v):
12         """
13         Add undirected edge between u and v.
14         """
15         self.graph.setdefault(u, []).append(v)
16         self.graph.setdefault(v, []).append(u)
17
18     def bfs(self, start):
19         """
20         Breadth-First Search traversal.
21         Uses queue.
22         """
23         visited = set()
24         queue = deque([start])
25         visited.add(start)
26
27         while queue:
28             vertex = queue.popleft()
29             print(vertex, end=" ")
30
31             for neighbor in self.graph.get(vertex, []):
```

```

47.py 2 ...
3 class Graph:
18     def bfs(self, start):
32         if neighbor not in visited:
33             visited.add(neighbor)
34             queue.append(neighbor)
35
36     def dfs(self, start, visited=None):
37         """
38         Depth-First Search traversal (recursive).
39         """
40         if visited is None:
41             visited = set()
42
43         visited.add(start)
44         print(start, end=" ")
45
46         for neighbor in self.graph.get(start, []):
47             if neighbor not in visited:
48                 self.dfs(neighbor, visited)
49 print("Graph Example:")
50 g = Graph()
51 g.add_edge(1, 2)
52 g.add_edge(1, 3)
53 g.add_edge(2, 4)
54 print("BFS starting from vertex 1:")
55 g.bfs(1)
56 print("\nDFS starting from vertex 1:")
57 g.dfs(1)
58 print()
59

```

Output:

```

Graph Example:
BFS starting from vertex 1:
1 2 3 4
DFS starting from vertex 1:
1 2 4 3

```

BFS vs DFS Comparison

Feature	BFS	DFS
Structure	Queue	Stack/Recursion
Traversal	Level-by-level	Depth-first
Shortest Path	Yes (unweighted)	No
Memory	Higher (wide graphs)	Lower (deep graphs)

Practical Use Cases

BFS:

- Shortest path

- Social network degree search
- Broadcasting systems

DFS:

- Cycle detection
- Path existence
- Topological